Code-Layout, Readability and Reusability

Date	1 November 2026
Team ID	[Enter Team ID]
Project Name	CrimeVision — Intelligent Multimodal Forensic Analysis Platform
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability

This review describes the structure and coding practices used in the current CrimeVision prototype. Ratings are conservative and should be confirmed against the final code submitted.

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing and indentation.	Partial	Review formatting across all files before submission.
2	Proper File Structure	Files are grouped by application responsibility.	Yes	Separate API, service, utility, static frontend, and configuration areas are present.
3	Meaningful Variable Names	Names communicate purpose.	Yes	Names such as register_case_evidence, list_evidence, and save_analysis describe their roles.
4	Function / Method Names	Functions are descriptively named.	Yes	Routes and helper functions generally use action-oriented names.
5	Code Comments	Comments explain non-obvious logic.	Partial	Module docstrings exist; add comments only where validation or processing logic is not self-explanatory.
6	Modular Design	Code is split into reusable modules.	Yes	Flask blueprints, case-store services, and upload utilities separate responsibilities.
7	No Redundant Code	Duplicate or unused code is minimized.	Partial	A final cleanup pass is recommended; this has not been independently verified across the whole repository.
8	Error Handling	Errors are handled and returned clearly.	Partial	Upload routes validate inputs and return HTTP errors; broader edge-case testing is still needed.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Case store service	Python	Case, evidence, and saved-analysis operations.	High
2	Case-management API	Flask	Create/list cases; list evidence and analyses; retrieve saved analysis.	High
3	Evidence registration API	Flask / Python	Registers supported image, video, and audio uploads against a case.	Medium
4	Upload utilities	Python	Shared file-saving and validation logic for supported media types.	Medium
5	Frontend API / UI helpers	JavaScript	Common requests, notices, and rendering of investigation records.	Medium

Overall Code Quality Assessment

Aspect	Rating (1–5)	Comments
Code Layout & Structure	4	Responsibilities are separated across API, service, utility, and frontend files.
Readability	3	Names are generally clear; consistent formatting and focused comments should be checked before handover.
Reusability	4	Case-store functions, Flask routes, and upload utilities support reuse across workflows.
Documentation / Comments	3	Module docstrings are present in key files; add concise usage notes for important workflows.
Overall Score	3.5 / 5	Practical prototype structure with room for cleanup and broader testing.

Overall Remarks

CrimeVision uses a practical modular layout for a student prototype. The backend separates API endpoints from case-storage and upload utilities, while the browser interface handles user interaction. The ratings above are a documentation-level assessment based on the project structure discussed; they are not the result of a complete automated code-quality audit.

Recommended final checks

1. Run the application and test the main case, evidence-upload, and saved-analysis workflows.
2. Remove unused imports or temporary debug code if found.
3. Keep comments focused on validation, file handling, and model-processing decisions.
4. Confirm that all reusable-module descriptions match the submitted source files.

Submission note: Replace [Enter Team ID]. Checklist ratings are intentionally conservative and should be adjusted if the final source review reveals differences.